

오늘의 작업기록 — 2026년 7월 16일

계약 문서화 — README에 에러 응답 형식 박기

(시간이 없는 날의 정직한 5분 — 7/11 원칙 2호)

한눈에

항목	내용
상황	시간이 아예 없음 + 잔디는 필요
판단	7/11 원칙 재적용 — 빈 커밋 금지, 대신 실질 있는 5분거리
한 일	README에 에러 응답 계약 문서화 (+14줄) — 어제 966c93f로 생긴 계약이 문서엔 없었다
커밋	54e3523 "document error response contract in README"
판정	grep -c "에러 응답": 작업 전 0(중복 없음) → 작업 후 1(들어감)
팁	heredoc 안 ``` 대신 4칸 들여쓰기(렌더링 동일, 폰 붙여넣기 안전)
다음	m106 CodeCrafters Redis 1단계 (프레시 스타트, 30~40분 확보 필요)

1. 오늘의 판단 — 원칙이 두 번째로 작동했다

7/11과 똑같은 상황이 왔다: 잔디는 찍고 싶고, 시간은 없다. 그날 세운 원칙 —

커밋 메시지는 공개 이력이다. 꿈수("defer", "keep streak")는 자백이 된다. 잔디는 결과이고, 실질이 없으면 비운다. 실질 있는 최소 단위를 찾을 수 있으면 그걸 한다.

오늘은 그 "실질 있는 최소 단위"가 명확히 존재했다: 어제 태어난 계약이 문서에 없었다. 966c93f가 만든 것은 코드만이 아니라 계약이다 — "이체 실패 시 이 형식의 JSON과 이 code들이 온다"는 약속. 그런데 README는 7/10 버전(핸들러 이전)이었다. 계약을 만들고 다음 날 문서화하는 건 실무에서도 자연스러운 흐름이라, 커밋 메시지도 정직하게 쓸 수 있었다: document error response contract in README — 한 일 그대로다.

2. 왜 문서화가 "실질"인가 — code 상수의 지위 변화

문서화 전과 후에 ACCOUNT_NOT_FOUND 같은 상수의 지위가 다르다:

	문서화 전	문서화 후
정체	구현 세부사항 (코드 안에만 존재)	공개된 계약 (README = 레포 첫 화면)
바꾸면	내 마음 (아무도 약속받지 않음)	파기 (문서를 믿고 만든 클라이언트가 깨짐)
보는 사람	코드를 열어본 사람만	레포에 들어온 모든 사람

어제 § 2에서 "code는 절대 안 바뀌는 계약"이라고 썼는데, 정확히는 문서에 박히는 순간부터 그렇다. 오늘 그 뜻을 박은 것이다. 커밋 이력도 채용담당자 관점에서 읽힌다: 기능 구현(966c93f) → 다음 날 계약 문서화(54e3523) — 만들고 끝이 아니라 쓰는 사람을 생각하는 흐름.

3. 절차 — 판정 → 추가 → 재판정 → 커밋

① 선판정 (중복 방지)

```
cd ~/ledger-api && grep -c "에러 응답" README.md # → 0
```

붙여넣기 전에 먼저 이미 있는지부터 확인 — 0이어야 진행. (>> 추가 모드는 무조건 뒤에 붙이므로, 두 번 실행하면 중복이 생긴다. 선판정이 그 방지책이다.)

② 추가 (>> heredoc — 7/11 도구 재사용)

```
cat >> README.md << 'EOF'

## 에러 응답 형식

이제 실패 시 HTTP 400과 함께 아래 JSON을 반환한다:

{"code": "INSUFFICIENT_BALANCE", "message": "잔액이 충분하지 않습니다"}

| code | 의미 |
| --- | --- |
| ACCOUNT_NOT_FOUND | 존재하지 않는 계좌 id |
| INVALID_AMOUNT | 금액이 0 이하 |
| INSUFFICIENT_BALANCE | 출금 계좌 잔액 부족 |

`code`는 기계 판별용 불변 상수(변경 안 됨), `message`는 사람에게 보여줄 문장(변경될 수 있음).
EOF
```

> (덮어쓰기)가 아니라 >> (추가)인 것 — 7/11에 기존 README를 날릴 뻔했던 그 지점에서 배운 도구 그대로다.

③ 재판정 + 커밋

```
grep -c "에러 응답" README.md # → 1 (0→1: 정확히 한 번 들어감)
git add README.md && git commit -m "document error response contract in README" && git push
# [main 54e3523] 1 file changed, 14 insertions(+)
# 966c93f..54e3523 main -> main
```

grep 0→1 패턴 — 작업 전 0(없음 확인), 작업 후 1(정확히 한 번 들어감 확인). 숫자 둘로 시작과 끝을 다 판정한다.

4. 작은 실전 팁 — heredoc 안의 ``는 피한다

원래 JSON 예시를 코드블록으로 쓰려 했는데, ****heredoc 안에** 가 있으면 폰 붙여넣기 과정에서 꼬일 위험이 있어 4칸 들여쓰기로 바꿨다. 마크다운에서 4칸 들여쓰기 = 코드블록**으로 렌더링이 동일하다. 폰 워크플로우의 제약(긴 붙여넣기 깨짐)을 아는 상태에서 도구를 고르는 것 — 환경에 맞춘 선택이다.

5. 로드맵 위치

- ledger-api: 코드(966c93f)와 문서(54e3523)가 이제 일치. 도메인 수정 여전히 0줄.
- 7월 잔여(15일): m106 CodeCrafters Redis → m107 월말 점검. 다음 세션 = m106 프레스 스타트(30~40분 확보하고).
- 8월 WAL 이유 3개 유지: 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

6. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 잔디가 필요한데 시간이 없을 때의 원칙은? 오늘 그 "실질 있는 최소 단위"는 왜 README 문서화였나?
2. 문서화 전과 후, code 상수의 지위는 어떻게 다른가? "계약"이 되는 시점은 언제인가?
3. 작업 전 grep -c가 0이어야 했던 이유는? >> 추가 모드와 어떤 관계인가?
4. > 와 >> 의 차이는? 이걸 어디서 배웠나?
5. heredoc 안에 ``를 안 쓰고 4칸 들여쓰기를 쓴 이유는? 렌더링은 어떻게 되나?
6. 커밋 메시지 "document error response contract in README"가 7/11 원칙에 비춰 정직한 이유는?

7. 회고

시간이 없는 날의 모양이 이제 정해졌다. 7/11에 한 번, 오늘 두 번째 — 잔디가 필요하면 빈 커밋이 아니라 실질 있는 최소 단위를 찾는다. 두 번 다 그 단위가 README였다는 것도 우연이 아니다: 코드는 세션이 필요하지만, **어제 만든 것의 문서화는 항상 정직한 다음 날 일감**이기 때문이다. 이 패턴은 기억해둘 만하다 — 앞으로도 시간 없는 날은 올 것이고, "어제 뭘 만들었는데 문서에 없나"가 첫 질문이 된다.

5분짜리였지만 남는 게 없진 않다. code 상수가 오늘부로 계약이 됐다는 것 — 어제는 코드 안의 문자열이었는데, 오늘 레포 첫 화면에 박히면서 바꾸면 파기가 되는 약속이 됐다. 그리고 grep 0→1 판정, heredoc 안 `` 회피 같은 잔기술도 손에 붙었다. 내일이든 언제든, 다음 세션은 m106이다 — 6379 포트를 내 손으로 여는 것. 온전한 시간 갖고 온다.

한 줄: 시간 없는 날의 정직한 5분 — 어제의 계약(code 3종)을 README에 박아 문서와 코드를 일치시켰다(54e3523, 0→1 판정). 7/11 원칙 2호 작동.